

설비 고장을 예측하는 AI 데이터 분석

튜플셋·딕셔너리 — 딕셔너리

딕셔너리 기초 → 순회와 갱신 → 자료구조 통합

posco

CONTENTS

목차

01 딕셔너리 기초 — 이름으로 값 찾기

02 순회와 갱신 — items·update·zip

03 자료구조 통합 — 종합 실습

posco

01

딕셔너리 기초

이름으로 값을 찾는 자료구조 · 만들기 · 키 조회 · 추가·수정·삭제 · get · in

이름으로 값을 찾는 딕셔너리

순서 번호 대신 이름표로 값을 바로 찾는 자료구조.

KEY MESSAGE 01

순서 번호 대신 **이름표**로 값을 바로 찾는 자료구조

KEY MESSAGE 02

센서 이름처럼 **이름이 분명한 데이터**에 가장 적합

리스트의 불편함과 딕셔너리의 등장

위치 관리의 부담을 키-값 짝지음으로 풀어낸다.

01 · 위치 관리

리스트의 한계

센서 빠지면 모든 번호가 **밀려** 다시 외워야 함

20개 센서 중 7번을 떼면 8번부터 전부 한 칸씩 밀린다.
외워둔 위치가 모두 어긋난다.

02 · 사전 비유

딕셔너리의 직관

단어를 찾으면 **뜻이 나오듯** 키로 값을 찾음

이름표를 찾으면 그 자리에 묶여 있는 값이
즉시 나오는 구조다.

03 · 키와 값

핵심 구조

이름표는 **키**, 연결된 값은 **밸류**로 짝지어
저장

키와 값이 짝을 이루는 게 딕셔너리의 핵심 —
key:value

딕셔너리 만들기

중괄호 안에 키와 값을 **콜론(:)**으로 짝지어 생성.

문법

```
{ 키 : 값 , 키 : 값 }
```

왼쪽이 이름표(키), 오른쪽이 그 값.

여러 쌍은 쉼표(,)로 구분.

키는 보통 문자열, 값에는 거의 무엇이든 가능.

셋과의 구분

콜론 유무로 셋과 구분

● 콜론(:) 있으면 **딕셔너리**

● 빈 중괄호 {} 는 **딕셔너리**

빈 셋은 `set()` 함수로 만들어야 한다.

딕셔너리 만들기 — 코드로 확인

키와 값을 콜론으로 짝지어 생성.

CODE · DICTIONARY

```
sensors = {"모터온도": 78, "진동": 0.5}
print(sensors)
print(type(sensors))
empty = {}
```

type 출력은 dict — 빈 중괄호는 빈 딕셔너리

키로 값 꺼내기

대괄호 안에 키를 넣어 연결된 값을 바로 꺼낸다.

문법

딕셔너리[키]

리스트에서 번호를 넣던 자리에 이름표를 넣는다.
위치가 바뀌어도 이름만 알면 정확히 찾아준다.
셋처럼 키 조회 속도가 매우 빠름.

위치를 몰라도 이름만 알면 됨

- 센서 100개여도 이름으로 즉시 조회
- 없는 키를 꺼내면 **KeyError**

안전한 조회는 잠시 뒤 `get` 으로 해결.

키로 값 꺼내기 — 코드로 확인

이름표로 값을 바로 조회.

CODE · DICTIONARY KEY

```
sensors = {"모터온도": 78, "진동": 0.5}
print(sensors["모터온도"])
print(sensors["진동"])
# 없는 키는 KeyError
```

위치 없이 이름만으로 조회 — 확실하지 않은 키는 다음 슬라이드의 `get` 으로

새로운 키-값 추가하기

없는 키에 값을 할당하면 그 쌍이 새로 추가된다.

문법

딕셔너리[새 키] = 값

리스트의 append 같은 별도 명령이 필요 없다.

빈 딕셔너리부터 시작해 데이터가 들어올 때마다 쌓는다.

같은 문법이 추가와 수정 모두 처리.

별도 명령 없이 이름표만 붙이면 됨

- 1 빈 딕셔너리부터 하나씩 쌓기 가능
- 같은 문법이 추가와 수정 모두 처리

새 키-값 추가 — 코드로 확인

새 이름표에 값을 붙여 추가.

CODE · DICTIONARY ADD

```
sensors = {"모터온도": 78}
sensors["펌프압력"] = 95
sensors["유량"] = 42
print(sensors)
```

빈 서랍장에 새 이름표 서랍을 다는 것과 같다 — 출력에 세 센서 모두 포함

기존 값 수정하고 삭제하기

있는 키에 값을 넣으면 수정, **del** 로 키-값 쌍을 삭제.

수정 문법

딕셔너리[**있는 키**] = 새 값

삭제 문법

del 딕셔너리[키]

추가할 때와 문법이 똑같다.

키 있으면 새로 만드는 대신 값만 바뀐다.

없는 키를 지우려 하면 오류 발생.

추가와 수정은 문법이 동일

1 같은 이름표에 최신값 덮어쓰기

● 삭제는 **del** 사용

측정할 때마다 값이 바뀌는 센서값은 항상 최신 상태 유지가 필요.

수정·삭제 — 코드로 확인

값 수정과 키-값 삭제.

CODE · DICTIONARY DELETE

```
sensors = {"모터온도": 78, "펌프압력": 95}
sensors["모터온도"] = 80
del sensors["펌프압력"]
print(sensors)
```

모터온도 78 → 80 갱신, 펌프압력 삭제 — 출력에는 모터온도 80만 남음

get으로 없는 키 안전하게 다루기

`get`은 없는 키를 꺼내도 오류 대신 기본값을 돌려준다.

문법

딕셔너리.`get`(키 , 기본값)

키 있으면 값, 없으면 오류 대신 `None`.

두 번째 인자로 기본값을 지정 가능.

여러 설비의 값을 더할 때 빠진 센서를 0 처리.

데이터가 들쭉날쭉할 때 안전장치

1 키 없으면 `None` 또는 지정한 기본값

● 일부 센서가 빠져도 계산 안 멈춤

어떤 설비엔 유량 센서가 있고 어떤 설비엔 없는 실무 상황의 안전장치.

get — 코드로 확인

없는 키도 기본값으로 안전 처리.

CODE · DICTIONARY GET

```
sensors = {"모터온도": 78}
print(sensors.get("모터온도"))
print(sensors.get("유량"))
print(sensors.get("유량", 0))
```

모터온도 → 78, 유량 → None, 유량 기본값 0 → 0 (오류 없음)

in으로 키 존재 확인하기

in 연산자로 특정 키가 딕셔너리에 있는지 확인.

문법

키 in 딕셔너리

있으면 True, 없으면 False.

조건문과 함께 쓰면 꺼내거나 삭제 전 안전성 확보.

get은 안전하게 꺼낼 때, in은 있는지만 확인할 때.

딕셔너리에서 in은 값이 아닌 키를 봄

1 키 있으면 True, 없으면 False

● 꺼내기·삭제 전 확인으로 오류 방지

값을 넣으면 False — 딕셔너리의 in은 항상 키를 본다.

in — 코드로 확인

키 존재를 확인 후 처리.

CODE · DICTIONARY IN

```
sensors = {"모터온도": 78, "펌프압력": 95}
print("모터온도" in sensors)
print(78 in sensors)
if "유량" in sensors:
    print(sensors.get("유량"))
```

모터온도 → True (키), 78 → False (값) — 딕셔너리의 in은 항상 키를 본다

딕셔너리 키가 될 수 있는 것

바뀌지 않는 값만 키가 될 수 있다 — 고정된 주소처럼 쓰이기 때문.

자료형	키 가능	이유
문자열	가능	안 바뀜 — "모터온도" 같이 한 번 정해지면 변경 불가
숫자	가능	안 바뀜 — 정수·실수 모두 키로 사용 가능
튜플	가능	안 바뀜 — 앞서 튜플이 키 될 수 있던 이유
리스트	불가	바뀜 — 이사 간 집으로 편지를 못 보내는 것과 같음

대부분 **문자열**을 키로 쓰니 평소엔 문제없다.

02

순회와 갱신

keys·values·items · update · zip · 중첩 딕셔너리 · 자료구조 선택

keys와 values로 키·값만 꺼내기

keys 는 모든 이름표를, **values** 는 모든 값을 한 번에 모은다.

문법

딕셔너리.**keys**()

딕셔너리.**values**()

한쪽 묶음만 떼어내 다룰 때 사용.

for로 하나씩 꺼낼 수 있고 list로 감싸 리스트로도 변환 가능.

keys로 꺼낸 이름을 셋으로 바꿔 비교에도 활용.

한쪽 묶음만 떼어내 다룰 때 사용

1 센서 이름 목록은 **keys**

측정값 합계·평균은 **values**

values로 sum과 len 조합 → 평균, keys로 정렬도 가능.

keys·values — 코드로 확인

키 목록과 값으로 평균 계산.

CODE · KEYS & VALUES

```
sensors = {"모터온도": 78, "압력": 95}
print(list(sensors.keys()))
print(list(sensors.values()))
avg = sum(sensors.values()) / len(sensors)
```

values의 합을 len으로 나누면 평균 — keys로 꺼낸 이름은 셋과 집합 연산도 가능

items로 키와 값 함께 꺼내기

items 는 키와 값을 짝으로 함께 꺼내 반복문에서 가장 많이 쓰인다.

문법

```
for key, value in 딕셔너리.items()
```

items는 키와 값을 튜플로 묶어 하나씩 건넨다.

앞서 배운 언패킹이 그대로 적용.

반복마다 이름과 값이 자동 분리됨.

앞서 배운 튜플 언패킹과 같은 패턴

1 for key, value in 딕셔너리.items()

이름과 값을 동시에 처리

딕셔너리에서 가장 많이 쓰는 패턴.

items — 코드로 확인

키와 값을 동시에 순회.

CODE · DICTIONARY ITEMS

```
sensors = {"모터온도": 78, "진동": 0.5}
for name, value in sensors.items():
    print(name, "측정값:", value)
```

첫 반복 name="모터온도", value=78 — 센서 이름과 값을 함께 다루는 거의 모든 작업의 출발점

len으로 항목 개수 세기

len은 딕셔너리에 담긴 키-값 쌍이 몇 개인지 세어준다.

문법

len(딕셔너리)

앞서 튜플 길이를 셀 때 쓴 그 도구.

키와 값은 항상 짝이니 키 개수가 곧 센서 수.

len은 리스트·튜플·셋·딕셔너리 모두에 동일하게 동작.

기대한 개수와 비교해 누락 확인

1 키 개수가 곧 등록된 센서 수

● 평균 계산에도 활용

센서 10개여야 하는데 len이 8이면 두 개가 빠졌다는 신호.

len — 코드로 확인

등록된 센서 수 확인.

CODE · DICTIONARY LEN

```
sensors = {"모터온도": 78, "진동": 0.5, "압력": 95}
print(len(sensors))
if len(sensors) < 5:
    print("센서 데이터 누락 확인 필요")
```

데이터를 받으면 가장 먼저 개수를 점검하는 좋은 습관 — len이 그 첫 검사를 담당

딕셔너리 순회 세 가지 방법

무엇이 필요한가 라는 질문으로 자연스럽게 정해진다.

방법	꺼내는 것	쓰는 경우
<code>keys</code>	키만	이름 목록 — 등록된 센서 이름 파악
<code>values</code>	값만	합계·평균 — 측정값 통계 계산
<code>items</code>	키와 값	함께 처리 — 이름과 값을 같이 다룸

음식점 주문서 비유 — 메뉴는 `keys`, 총 인분은 `values`, 메뉴별 인분은 `items`.

순회 패턴 정리 — 핵심

무엇이 필요한가로 결정 — 이름만, 값만, 아니면 둘 다.

KEY MESSAGE 01

무엇이 필요한가로 결정 — 이름만, 값만, 아니면 둘 다

KEY MESSAGE 02

실무에서는 이름과 값을 함께 쓰는 `items` 가 가장 많이 쓰임

순회하며 임계값과 비교하기

items 순회와 조건문을 결합해 기준치를 넘는 센서를 가려낸다.

설비 모니터링의 핵심

각 센서가 정상 범위인지 확인

임계값은 넘으면 안 되는 기준선.

센서마다 다르니 임계값도 딕셔너리로 관리.

같은 센서 이름을 다리 삼아 두 딕셔너리를 연결.

측정값과 임계값을 각각 딕셔너리로

1 같은 센서 이름으로 둘을 비교

● 설비 이상 감지의 기본 형태

이상 탐지를 본격적으로 배울 때도 출발점은 이 작업.

임계값 비교 — 코드로 확인

측정값과 임계값 비교.

CODE · DICTIONARY COMPARISON

```
values = {"모터온도": 95, "압력": 88}
limits = {"모터온도": 90, "압력": 90}
for name, value in values.items():
    if value > limits[name]:
        print(name, "경고")
```

모터온도 $95 > 90 \rightarrow$ 경고, 압력 $88 \leq 90 \rightarrow$ 통과 — 같은 이름이 두 딕셔너리를 연결

update로 여러 값 한 번에 갱신하기

update 는 여러 키-값을 한 번에 추가하거나 갱신.

문법

딕셔너리.**update**(새 딕셔너리)

새 딕셔너리의 키가 기존에 있으면 갱신, 없으면 추가.
갱신과 추가가 동시에 일어남.
센서 50개여도 한 번이면 됨.

있는 키는 갱신, 없는 키는 추가

1 새 측정 묶음을 통째로 반영

● 센서 50개도 한 줄로 처리

설비에서 새 측정 묶음이 한꺼번에 들어올 때 한 줄로 반영.

update — 코드로 확인

갱신과 추가를 한 번에.

CODE · DICTIONARY UPDATE

```
sensors = {"모터온도": 78, "진동": 0.5}
new_data = {"모터온도": 80, "유량": 42}
sensors.update(new_data)
print(sensors)
```

모터온도 78→80 갱신, 유량 새로 추가 — 한 명령으로 갱신·추가 동시 처리

zip으로 두 리스트를 딕셔너리로 만들기

zip은 이름 리스트와 값 리스트를 짝지어 딕셔너리로 묶는다.

문법

`dict( zip( 이름, 값 ) )`

지퍼처럼 두 리스트를 앞에서부터 하나씩 짝지음.

dict로 감싸면 딕셔너리.

파일에서 읽은 데이터를 정리할 때 유용.

두 리스트 순서가 정확히 맞아야 함

1 dict(zip(이름, 값)) 형태

● 지퍼처럼 앞에서부터 하나씩 짝지음

어긋나면 엉뚱한 이름에 엉뚱한 값이 붙는다.

zip — 코드로 확인

두 리스트를 딕셔너리로 결합.

CODE · DICTIONARY ZIP

```
names = ["모터온도", "진동", "압력"]
values = [78, 0.5, 95]
sensors = dict(zip(names, values))
print(sensors)
```

첫째끼리·둘째끼리 짝지음 — 모터온도:78, 진동:0.5, 압력:95

딕셔너리 안에 리스트·튜플 담기

딕셔너리 값에 리스트나 튜플을 넣어 센서 하나에 여러 값 저장.

문법

```
{ "이름": [v1, v2, v3] }  
sensors[ "이름" ][ 0 ]
```

센서 이름이라는 키 아래에 측정값 리스트를 매단다.

꺼낼 때 리스트를 꺼내고, 인덱스를 한 번 더.

앞서 배운 중첩 인덱싱과 동일.

꺼낼 때는 인덱스를 한 번 더 사용

1 한 센서의 시간별 측정 이력 관리

고정 정보는 튜플로 안전하게

모터온도를 매분 측정하면 값이 60개 — 리스트에 매달면 한 키에 다 담긴다.

값에 리스트 담기 — 코드로 확인

센서별 측정 이력으로 통계.

CODE · DICTIONARY LIST

```
sensors = {"모터온도": [78, 79, 80]}
temps = sensors["모터온도"]
print(sum(temps) / len(temps))
print(max(temps))
```

sum/len → 평균 79, max → 최고값 80 — 단순 이름-값을 넘어 풍부한 데이터 표현

설비별 센서 데이터 구조 설계하기

여러 정보를 가진 설비 데이터는 구조를 미리 설계하고 담는다.

설계의 원칙

담기 전에 구조를 먼저

이름·센서·측정값·상태가 다 섞여 있으니 설계 먼저.

서랍에 양말 칸, 속옷 칸을 나누듯이.

좋은 설계의 기준 — 원하는 정보를 쉽게 찾을 수 있는가.

자주 찾는 정보를 키로 삼기

1 무엇을 키로, 무엇을 값으로 정하기

원하는 정보를 쉽게 찾는 구조

센서 측정값을 자주 찾으면 센서 이름을 키로 삼는다.

구조 설계 — 코드로 확인

설비 정보를 딕셔너리로 설계.

CODE · DICTIONARY DATA

```
motor = {  
    "name": "1번 모터",  
    "온도": [78, 79, 80],  
    "status": "가동중",  
}
```

name·온도·status를 키·값으로 표현 — Pandas로 표를 다룰 때로 이어지는 관점

딕셔너리 안의 딕셔너리 다루기

딕셔너리 값에 또 딕셔너리를 넣어 여러 설비를 체계적으로 관리.

문법

```
plant[ 설비 ][ 센서 ]
```

설비 딕셔너리들을 큰 딕셔너리에 담는다.

설비 이름이 키, 그 설비 정보 딕셔너리가 값.

안쪽 값을 꺼낼 땐 키를 두 번.

안쪽 값은 키를 두 번 써서 접근

- 1 설비 이름을 키, 설비 정보를 값으로
- 바깥에서 안쪽 순서로 찾아 들어감

앞서 배운 중첩 튜플과 같은 원리 — 번호 대신 이름을 두 번.

중첩 딕셔너리 — 코드로 확인

키를 두 번 써서 안쪽 값 꺼내기.

CODE · DICTIONARY KEYS

```
plant = {  
    "1번모터": {"온도": 78, "상태": "정상"},  
    "2번펌프": {"압력": 95, "상태": "경고"},  
}  
print(plant["2번펌프"]["압력"])
```

2번펌프 → 압력 95 — items로 순회하면 모든 설비를 하나씩 점검 가능

튜플·셋·딕셔너리 선택 기준

데이터를 보고 세 질문을 던지면 대부분 정해진다.

자료구조	특징	적합한 데이터
튜플	안 바뀜	고정 묶음 — 센서 이름·값, 위치 좌표
셋	중복 없음	종류·비교 — 두 목록 비교, 종류 세기
딕셔너리	이름 조회	이름-값 짝 — 센서 이름으로 측정값 찾기

세 질문 — 안 바뀌나 · 중복 제거나 비교가 필요한가 · 이름으로 찾나

자료구조 선택 — 핵심

고정된 묶음은 튜플, 중복 없는 목록과 비교는 셋, 이름 조회는 딕셔너리.

KEY MESSAGE 01

고정된 묶음은 튜플, 중복 없는 목록과 비교는 셋

KEY MESSAGE 02

이름으로 찾는 데이터는 딕셔너리로 선택

실제로는 조합 — 튜플을 리스트에, 딕셔너리 값에 리스트또 딕셔너리

자료구조 학습 전체 흐름

튜플로 묶고, 셋으로 거르고, 딕셔너리로 찾고, 조합해 관리.

STEP 1

튜플로 묶기

짜지어진 데이터를 튜플로 묶어
어긋남 방지.



STEP 2

셋으로 중복 제거

중복을 거르고 두 목록을 비교.



STEP 3

딕셔너리로 조회

이름을 붙여 값을 바로 찾는다.



STEP 4

조합으로 관리

셋을 조합해 센서 데이터를
깔끔하게 관리.

출발점은 짜지어진 데이터가 따로 있으면 어긋난다는 문제 — 묶고 거르고 찾으면 복잡한 센서 데이터도 정돈된다

자료구조 종합 정리 — 핵심

데이터 성격에 맞는 자료구조를 고르면 깔끔하게 관리된다.

KEY MESSAGE 01

데이터 성격에 맞는 자료구조를 고르면 **깔끔하게 관리됨**

KEY MESSAGE 02

이 감각은 **NumPy·Pandas** 등 모든 데이터 분석의 기초

03

자료구조 통합 — 실습

딕셔너리 만들기·갱신·통계·분류·csv 처리·미니 리포트·종합

실습 1. 딕셔너리 만들고 다루기

센서명을 키, 측정값을 값으로 하는 딕셔너리를 만들고 추가·수정·안전 조회하기

단계

- ① 센서명을 키, 측정값을 값으로 딕셔너리 저장
- ② 키로 값을 꺼내고 새 키로 추가·기존 키로 수정
- ③ get으로 없는 키를 기본값으로 조회, in으로 키 존재 확인

예상 결과

78
추가·수정된 딕셔너리
0 (없는 키)
True

실습 2. update로 여러 값 한 번에 갱신하기

update로 여러 항목을 한 번에 갱신하고 삭제·개수 확인하기

단계

- ① 센서 딕셔너리와 새 데이터 딕셔너리를 각각 저장
- ② update로 새 데이터를 한 번에 반영(있으면 수정, 없으면 추가)
- ③ del로 특정 키를 삭제하고 len으로 개수 확인

예상 결과

갱신된 딕셔너리
센서 수: 2

실습 3. 딕셔너리로 통계 내기

values로 평균을, items 순회로 최댓값 센서를 구하기

단계

- ① 센서명-측정값 딕셔너리 저장
- ② values의 합을 개수로 나눠 평균 구하기
- ③ items로 순회하며 가장 큰 값과 그 센서명을 찾아 출력

예상 결과

평균: 76.7
최댓값 센서: 압력 95

실습 4. zip으로 센서명-값 매핑하기

zip으로 두 리스트를 짝지어 딕셔너리로 만들기

단계

- ① 센서명 리스트와 측정값 리스트를 각각 저장
- ② zip으로 두 리스트를 짝지어 dict로 변환
- ③ items로 순회하며 이름-값 쌍 출력

예상 결과

```
{'온도': 78, '진동': 0.5, '압력': 95}
각 이름-값 쌍 출력
```


실습 5. 임계값으로 경고 센서 분류하기

측정값과 임계값 딕셔너리를 비교해 경고 센서를 모으기

단계

- ① 측정값 딕셔너리와 임계값 딕셔너리를 각각 저장
- ② items로 순회하며 각 센서 값이 같은 이름의 임계값을 넘는지 비교
- ③ 넘는 센서 이름을 빈 리스트에 모아 출력

예상 결과

경고 센서: ['온도']

실습 6. 중첩 디렉터리로 설비 관리하기

디렉터리 안에 디렉터리를 담고 중첩 키로 접근·조건 검색하기

단계

- ① 설비명을 키로, 각 설비 정보(디렉터리)를 값으로 하는 중첩 디렉터리 저장
- ② 중첩 키로 특정 설비의 특정 값을 꺼내기
- ③ items 순회로 상태가 "경고"인 설비만 찾아 출력

예상 결과

95
2번펌프 점검 필요

실습 7. 표 데이터를 딕셔너리로 변환하기

행 문자열을 `split`으로 나눠 딕셔너리로 만들기

단계

- ① 한 줄에 "센서명,측정값" 형태인 행 문자열들을 리스트로 저장
- ② `for`로 각 행을 쉼표로 `split`해 이름과 값으로 나누기
- ③ 이름을 키, 값을 숫자로 바꿔 딕셔너리에 추가

예상 결과

```
{'온도': 78.0, '압력': 95.0, '진동': 0.5}
```

실습 8. 센서 데이터 통합 정리

딕셔너리로 평균을 구하고 위험 센서를 셋으로 모아 정렬 출력하기

단계

- ① 센서 측정값 딕셔너리와 임계값 딕셔너리 저장
- ② values로 전체 평균 구하기
- ③ items 순회로 임계값 초과 센서를 셋에 모으기
- ④ 셋을 정렬해 출력

예상 결과

평균: 95.5

위험 센서: ['압력', '온도']